Machine Learning and Inductive Inference - Lecture 6: Evaluation (5 Nov)

Lecture overview

Lecture 6 establishes evaluation within inductive inference, focusing on how models generalize from observed data to unseen instances through metrics for classification, regression, and probability estimation. The session contrasts basic measures like accuracy and error rate with more robust frameworks such as the confusion matrix, which distinguishes between true/false positives and negatives to reveal specific error patterns. A critical exam concept is the sensitivity of accuracy to class skew; in imbalanced datasets, high accuracy can mask total failure to detect minority classes, necessitating alternatives like balanced accuracy or cost-sensitive evaluation that integrates asymmetric penalties for false positives and false negatives. The lecture details the construction and interpretation of ROC curves, where the convex hull identifies optimal classifiers under linear costs, and defines AUC as the probability that a random positive is ranked higher than a random negative. However, ROC curves can be misleading under severe class imbalance, prompting the introduction of Precision-Recall (PR) curves, which are preferred for information retrieval and anomaly detection despite lacking the linear interpolation properties found in ROC space.

The second half of the lecture addresses evaluation methodology, emphasizing the necessity of independent training, validation, and test sets to prevent data leakage and ensure true generalization. It explores the data dilemma between maximizing training set size for model quality and test set size for reliable error estimation, proposing cross-validation as a solution that partitions data into k folds to maximize utility while maintaining independence. The session distinguishes between internal and nested cross-validation strategies for hyperparameter tuning, framing the selection of parameters like tree depth or regularization strength as a search problem solvable via grid or random search. Crucially, the lecture warns against data leakage caused by global preprocessing or temporal dependencies, mandating that steps like feature selection be repeated within each fold. Finally, it covers classifier calibration using techniques like isotonic regression and introduces regression metrics such as RMSE, MAE, and relative error to quantify performance across diverse deployment contexts.

Slide 1: Lecture 6: Evaluation

Lecture 6 covers how to evaluate learned models: choose measures that reflect the task, estimate performance on data kept out of fitting, and avoid letting class proportions or tuning distort the result. The deck is based on Jesse Davis’s Principles of Machine Learning slides.

Slide 2: - Metrics

The lecture sets three goals: understand classification, regression, and probability metrics; know what ROC and precision-recall (PR) curves show and how to construct them; and match a measure to the real problem. “Know their behaviour” means understand what changes each score and when it can mislead, rather than memorize a score without its assumptions.

Slide 3: - Metrics

The second goal is a fair evaluation workflow: cross-validation, independent training/validation/test roles, and common feature transformations and feature-selection methods. Keep the test set out of model fitting and tuning so it can provide a final estimate on unseen examples.

Slide 4: Evaluation metrics

Section divider for evaluation metrics. The following slides build from accuracy and error rate to confusion matrices, class-skew effects, cost-sensitive decisions, ROC and PR curves, probability calibration, and regression error measures.

Slide 5: Slide 5

This is the running classification example: each point is a song plotted by beats per minute (BPM) and energy. Green pluses and red minuses are the two true classes for the playlist task. The points overlap, so no single boundary will classify every song correctly.

Slide 6: M1

The same song data now has two candidate decision boundaries, M1 and M2. Each line assigns examples on either side to a class; because the boundaries cut through different points, the models make different errors. The slide asks how to compare them, not which line is universally best.

Slide 7: M1

The first evaluation question is comparative: which of M1 and M2 is better? A visual boundary alone cannot answer it; we need observed labels and a measure of the errors that matter for the task.

Slide 8: M1

The question becomes both relative and absolute: which model is better, and how good is each? A ranking between two candidates is not the same as knowing whether either meets a useful performance target.

Slide 9: M1

The song example leads to the central measurement question: how should performance be measured? Subsequent slides show that the answer depends on the outcome distribution, error costs, and whether the model outputs labels or scores/probabilities.

Slide 10: - Two natural performance measures for classification: Error rate and accuracy

Two fundamental performance measures for evaluating classification models are error rate and accuracy. A prediction is defined as a success when the model correctly identifies an instance's class, whereas it constitutes an error if the predicted class is incorrect. These metrics provide a direct assessment of how often a classifier succeeds or fails in assigning the correct labels to data instances.

Slide 11: - Two natural performance measures for classification: Error rate and accuracy

Two fundamental performance measures for classification are error rate and accuracy. A prediction is defined as a success when the instance's class is predicted correctly, whereas an error occurs when the prediction is incorrect. Accuracy is calculated by dividing the total number of successes by the total number of instances.

Slide 12: - Two natural performance measures for classification: Error rate and accuracy

Two fundamental performance measures for classification are accuracy and error rate, which quantify prediction outcomes based on individual instances. A success occurs when an instance's class is predicted correctly, while an error denotes an incorrect prediction. Accuracy is calculated as the ratio of successes to total instances, whereas the error rate is the ratio of errors to total instances.

Slide 13: - Two natural performance measures for classification: Error rate and accuracy

Two fundamental performance measures for classification are accuracy and error rate, defined by whether an instance's class is predicted correctly (success) or incorrectly (error). Accuracy is calculated as the ratio of successes to total instances, while the error rate is the ratio of errors to total instances. These metrics are mathematically complementary, meaning accuracy equals one minus the error rate.

Slide 14: - Two natural performance measures for classification: Error rate and accuracy

To quantify classification performance beyond vague assertions, two natural measures are used: accuracy and error rate. Accuracy represents a positive view by calculating the ratio of correct predictions (successes) to total instances, while error rate offers a negative view by measuring incorrect predictions; mathematically, accuracy equals one minus the error rate. Comparing models M1 and M2 illustrates that higher accuracy corresponds to more successes and fewer errors, providing a concrete metric for model evaluation.

Slide 15: Predicted True

A confusion matrix, also known as a contingency table, is used to evaluate binary classification models by organizing predictions into four categories: predicted true, predicted false, actually true, and actually false. This structure allows for a clear comparison between the model's output and the actual ground truth labels.

Slide 16: Predicted True

Slide 16 introduces the confusion matrix, also known as a contingency table, used to evaluate binary classification models. It specifically identifies the "True Positive" (TP) outcome, which occurs when an instance is actually true and correctly predicted as true, illustrated here with a count of 8. This framework organizes predictions into categories based on their alignment with actual values to assess model performance.

Slide 17: Predicted True

Slide 17 introduces the confusion matrix, also known as a contingency table, used to evaluate binary classification models. The example provided illustrates specific outcomes where an "Actually True" case correctly predicted as true results in a True Positive (TP) count of 8, while an "Actually False" case correctly predicted as false yields a True Negative (TN) count of 12. These metrics form the foundation for assessing prediction accuracy within this classification framework.

Slide 18: Predicted True

Slide 18 presents a confusion matrix, also known as a contingency table, used to evaluate binary classification performance. The table categorizes predictions into four outcomes based on actual versus predicted labels: True Positive (TP) with 8 instances, False Negative (FN) with 3 instances, and True Negative (TN) with 12 instances. This structure visually organizes the results to distinguish between correct classifications and specific types of errors within the dataset.

Slide 19: Predicted True

A confusion matrix (or contingency table) evaluates binary classification by subdividing predictions into four categories: True Positives (correctly predicted positives), True Negatives (correctly predicted negatives), False Negatives (actual positives misclassified as negative), and False Positives (actual negatives misclassified as positive). This structure reveals model performance, where a worst-case scenario predicts only false positives with no true positives, while an ideal case contains only true positives. To construct this for a model outputting numerical probabilities, values are sorted from high to low, and boundaries between true labels are located to determine…

Slide 20: Predicted True

Slide 20 defines a confusion matrix for binary classification, categorizing outcomes into True Positives (8), False Negatives (3), False Positives (4), and True Negatives (12). Accuracy is calculated as the sum of correct predictions divided by the total instances, while the error rate represents incorrect predictions; notably, accuracy equals one minus the error rate. Although the transcript discusses shifting classification boundaries to generate true positive rates and false positives, these specific threshold examples are not reflected in the static values presented on this slide.

Slide 21: Multiclass confusion matrix

These are four-class confusion matrices for genre prediction with 3-NN (left) and XGBoost (right). Rows are true genres and columns are predicted genres; diagonal cells are correct predictions and off-diagonal cells show which genres are confused. For example, the 3-NN model often confuses Classical with Old-Time/Historic (10 cases), while XGBoost has a stronger diagonal overall but still confuses some Pop and Hip-Hop examples. In a multiclass matrix, diagonal counts are not “true positives plus true negatives” as in the binary accuracy formula; sum the diagonal for correct predictions.

Slide 22: - Let class skew be ☐=

Class skew (☐) represents the proportion of positive instances in the dataset. Accuracy is defined as a weighted average of the True Positive Rate (TPR) and True Negative Rate (TNR), calculated as Acc = ☐ × TPR + (1 − ☐) × TNR. This relationship demonstrates that accuracy is highly sensitive to class skew, meaning the distribution of classes directly influences the overall performance metric.

Slide 23: - Let class skew be ☐=

Accuracy is a weighted average of the true positive rate (TPR) and true negative rate (TNR), heavily influenced by class skew (π). As demonstrated, with fixed TPR=0.6 and TNR=0.8, accuracy drops from 0.7 to 0.62 when skew increases from 0.5 to 0.9, proving its sensitivity to imbalanced datasets. Consequently, research often employs balanced accuracy by fixing $\pi=0.5$ regardless of actual distribution to mitigate this bias.

Slide 24: - Is 99% accuracy good?

Slide 24 introduces the critical question of whether 99% accuracy is sufficient, highlighting that high accuracy alone can be misleading. This section serves as a transition to discuss specific problems associated with relying solely on accuracy as a performance metric in classification tasks.

Slide 25: - Is 99% accuracy good?

99% accuracy is not inherently good because it can mask a model that fails to learn anything in highly imbalanced datasets. While one model correctly identifies most true and false cases, another achieves the same 99% score by simply predicting "false" for every instance when positives are rare (e.g., only 20 out of 2000). This demonstrates that accuracy alone is insufficient to evaluate performance when class skew exists, as a trivial strategy can yield deceptively high scores without capturing actual patterns.

Slide 26: Problems with accuracy

The two boundaries make different errors on the same BPM-energy points. M1 and M2 can nevertheless have the same total number of correct classifications, so equal accuracy may hide which examples each model gets wrong.

Slide 27: Problems with accuracy

The slide states that M1 and M2 have the same accuracy, yet some settings prefer one. Their decision boundaries trade one set of mistakes for another. The next slides make that preference explicit by assigning different costs to false positives and false negatives.

Slide 28: - Suppose we have misclassification costs

This section introduces the concept of incorporating misclassification costs into decision-making, specifically defining C_{FP} as the cost of a false positive and C_{FN} as the cost of a false negative. These parameters are essential for evaluating scenarios where different types of errors carry distinct penalties, moving beyond simple accuracy metrics to account for the specific financial or operational impact of incorrect predictions.

Slide 29: - Suppose we have misclassification costs

A false positive and a false negative can have different consequences. The expected cost of one prediction is the cost of each error type multiplied by its probability, then summed. This is why the cheapest model under accuracy need not minimize the real-world loss.

Slide 30: - Suppose we have misclassification costs

When misclassification costs exist, the expected cost of a prediction combines the cost of false negatives (C_{FN}) and false positives (C_{FP}), weighted by class prevalence (π). This theoretical expectation is approximated using the False Negative Rate (FNR) and False Positive Rate (FPR), where FNR represents missed positive cases and FPR represents incorrectly flagged negative cases. The resulting formula, $C = C_{FN} \times \pi \times \text{FNR} + C_{FP} \times (1 - \pi) \times \text{FPR}$, quantifies the total expected loss by integrating these error rates with their specific financial or operational penalties.

Slide 31: - Suppose we have misclassification costs

When misclassification costs exist, the expected cost of a prediction combines the cost of false negatives (C_{FN}) and false positives (C_{FP}), weighted by class prevalence (π) and error rates. This cost is approximated using the False Negative Rate (FNR) and False Positive Rate (FPR) rather than raw probabilities, as accuracy alone cannot compute this expected cost. The formula explicitly accounts for the distinct financial or operational penalties associated with each type of error.

Slide 32: - Receiver Operating Characteristic curves

Originating from WWII radar research, Receiver Operating Characteristic (ROC) curves are a visualization tool specifically for binary classification problems. These curves assess model performance by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR). The resulting graph allows for the comparison of different models and distinguishes their effectiveness from random guessing based on their position relative to the diagonal baseline.

Slide 33: - Receiver Operating Characteristic curves

A ROC curve plots true-positive rate (TPR) against false-positive rate (FPR) as the threshold changes. The diagonal is random guessing; curves closer to the upper-left generally give more true positives at a given false-positive rate. The endpoint at (0,0) predicts every case negative, while (1,1) predicts every case positive; these extreme rules are not useful classifiers by themselves.

Slide 34: - Given: numeric outputs for each test example (e.g. ☐(☐= 1|☐))

To construct a Receiver Operating Characteristic (ROC) curve given numeric outputs like $P(y=1|x)$ for test examples, first sort these values and identify boundaries where predicted labels change. Calculate the False Positive Rate (FPR) and True Positive Rate (TPR) at each boundary, then plot these coordinate pairs. Finally, connect the plotted points to form the complete ROC curve, which visualizes classifier performance across varying decision thresholds.

Slide 35: Predicted

The table gives ten predicted scores with their true labels: five positives and five negatives. This is the unsorted starting data for a ROC example. The graph axes are FPR horizontally and TPR vertically, each ranging from 0 to 1; the curve will be built by moving the decision threshold through the scores.

Slide 36: Predicted

Sort the examples by score, highest first: 0.95+, 0.93+, 0.73+, 0.63−, 0.61−, 0.52+, 0.37−, 0.33+, 0.12−, 0.05−. With five positives and five negatives, each newly included positive raises TPR by 0.2 and each negative raises FPR by 0.2.

Slide 37: Predicted

For each threshold, predict positive for scores at or above it and count TP and FP. Divide TP by the five actual positives to get TPR, and FP by the five actual negatives to get FPR. Plot each resulting (FPR, TPR) pair; lowering the threshold adds examples and traces the staircase from (0,0) to (1,1).

Slide 38: Predicted

As the threshold moves down through the three highest scores (0.95+, 0.93+, 0.73+), three of five positives are found and no negatives are admitted. The ROC point is therefore (FPR=0, TPR=0.6); the earlier all-negative rule is (0,0).

Slide 39: Predicted

The next scores, 0.63− and 0.61−, add two false positives but no true positives. FPR rises from 0 to 0.4 while TPR remains 0.6, moving horizontally to (0.4, 0.6).

Slide 40: Predicted

Including the next score, 0.52+, adds a fourth true positive without another negative. TPR rises to 0.8 and FPR stays 0.4, giving (0.4, 0.8).

Slide 41: Predicted

Including 0.37− adds a third false positive. The ROC point moves horizontally to (FPR=0.6, TPR=0.8); this threshold change improves coverage of negatives only by incorrectly flagging another one.

Slide 42: Predicted

Including 0.33+ finds the fifth and final positive. TPR reaches 1.0 while FPR remains 0.6, giving (0.6, 1.0). The two lower-scored negatives are still excluded.

Slide 43: Predicted

Finally, lowering the threshold to include 0.12− and 0.05− admits all remaining negatives. FPR reaches 1.0 while TPR stays 1.0. The complete staircase shows the sensitivity/false-alarm trade-off across thresholds.

Slide 44: Convex hull of a ROC curve

The ROC convex hull connects the non-dominated operating points. For a fixed class prevalence and error-cost ratio, a point below the hull is worse than some point on it. Randomly choosing between two classifiers realizes expected rates along the line segment between their ROC points, so a hull segment can be useful when no single threshold gives the desired trade-off.

Slide 45: - Two (FPR, TPR) points

To achieve a target operating point of (FPR=0.2, TPR=0.7) between two classifiers M1 (0.0, 0.6) and M2 (0.8, 1.0), one can employ randomized selection by flipping a biased coin. By choosing M1 with probability 0.75 and M2 with probability 0.25, the resulting performance linearly interpolates between the two points in ROC space. This technique demonstrates that randomization allows for the construction of any intermediate point along the line segment connecting two distinct classifiers.

Slide 46: - Area under the ROC curve (AUC)

AUC summarizes the ROC ranking performance. It equals the probability that a randomly chosen positive example receives a higher score than a randomly chosen negative one, with half credit for tied scores. It does not select a decision threshold or encode the application’s error costs.

Slide 47: - Area under the ROC curve (AUC)

An AUC of 1 means every positive is ranked above every negative, so some threshold separates them perfectly; 0.5 is the expected AUC of random ranking. A larger AUC summarizes better ranking over thresholds, but does not guarantee better performance at the particular threshold or operating region a task needs.

Slide 48: - ROC can be misleading if:

When positives are rare, a small FPR can still mean many false alarms because it is applied to a very large negative class. ROC rates normalize by each class and can obscure that absolute count. PR curves make the yield among predicted positives visible; the choice of curve should follow the task and class prevalence, not a blanket rule.

Slide 49: - Primarily arise out of the literature on information retrieval (web search)

PR curves come from information retrieval, where finding relevant items among many non-relevant ones is central. Precision is the share of predicted positives that are truly positive; recall is the share of all positives that are found. The horizontal baseline for a no-skill ranking is the positive-class prevalence, so PR curves change when prevalence changes.

Slide 50: - Primarily arise out of the literature on information retrieval (web search)

Changing the score threshold creates a precision-recall trade-off: a strict threshold returns fewer predictions, often with higher precision but lower recall; relaxing it usually finds more positives while adding false positives and reducing precision. The useful operating point depends on the relative cost of missed positives and false alarms.

Slide 51: - What happens when we interpolate? ([Goadrich, et al., ILP 2004])

Linear interpolation in ROC space corresponds to randomizing between classifiers, because expected TPR and FPR mix linearly. In PR space, recall and precision are ratios computed from TP and FP counts, so mixing the counts generally maps to a curved path rather than the straight line between the PR points. Do not assume that a straight PR segment represents a realizable classifier.

Slide 52: - What happens when we interpolate? ([Goadrich, et al., ILP 2004])

For the first operating point, TP=500 and FP=500. With 1,000 positives, recall/TPR is 0.50; with 8,000 negatives, FPR is 500/8,000≈0.06. Precision is TP/(TP+FP)=500/1,000=0.50. The slide is filling one set of counts into both ROC and PR measures.

Slide 53: - What happens when we interpolate? ([Goadrich, et al., ILP 2004])

The completed first row confirms the distinction: TP=500, FP=500 gives recall 0.50 and precision 0.50, even though the ROC FPR is only about 0.06 because there are many negatives. Precision is not FPR; they use different denominators.

Slide 54: - What happens when we interpolate? ([Goadrich, et al., ILP 2004])

The table compares two thresholds on a dataset with 1,000 positives and 8,000 negatives: (TP=500, FP=500) gives recall 0.50 and precision 0.50; (TP=1,000, FP=9,000) gives recall 1.00 and precision 0.10. The first point has not yet shown the interpolated middle threshold.

Slide 55: - What happens when we interpolate? ([Goadrich, et al., ILP 2004])

The middle threshold adds 250 true positives and 4,250 false positives, moving counts from (500,500) to (750,4,750). This is a midpoint in counts, not a midpoint in precision: the resulting precision is 750/(750+4,750)≈0.136, displayed as 0.14 when rounded.

Slide 56: - What happens when we interpolate? ([Goadrich, et al., ILP 2004])

At the middle point, TP=750 and FP=4,750 give recall 0.75 and FPR 0.53 (4,750/8,000). Precision is about 0.14. Thus the ROC coordinates are halfway between the endpoint coordinates, while the PR coordinates are not the simple arithmetic midpoint of (0.50,0.50) and (1.00,0.10).

Slide 57: - What happens when we interpolate? ([Goadrich, et al., ILP 2004])

The three rows make the count-based interpolation explicit: (500,500) → (750,4,750) → (1,000,9,000). Recall/TPR rises 0.50→0.75→1.00; FPR rises about 0.06→0.53→1.00; precision is 0.50→about 0.14→0.10. Precision falls nonlinearly because its denominator is TP+FP.

Slide 58: Interpolation in PR space

With 20 positives and 2,000 negatives, point A has TP=5, FP=5, recall 0.25, precision 0.50. Adding one true and five false positives at each step yields the table through point B (TP=10, FP=30, recall 0.50, precision 0.25). Each row is a feasible threshold operating point on this fixed dataset.

Slide 59: Interpolation in PR space

The plotted points form a curved PR path from A to B. The dotted straight segment is ordinary linear interpolation between the endpoints; its intermediate precision values do not match the feasible points computed from the table's TP/FP counts. This visualizes why PR interpolation cannot be treated like linear ROC interpolation.

Slide 60: Problem 2: Changes in skew

This slide compares PR curves at three class prevalences while holding the same recall value (75%). Precision is 21% with 10% positives, 50% with 30% positives, and 70% with 50% positives. More positives among the same set of predicted positives increase precision; the dashed baseline equals prevalence. Precision-recall performance therefore depends on class skew.

Slide 61: - In ROC space, all models are possible

For a fixed dataset, TP, FP, and the class totals must be valid counts. ROC rates can represent any feasible confusion-matrix operating point (and randomized classifiers give expected points between thresholds), whereas the fixed positive/negative totals restrict which precision-recall pairs are possible. PR space can therefore contain mathematically unachievable regions.

Slide 62: - Key Insight: ROC and PR curves are constructed from a fixed dataset

With P=100 positives and N=1,000 negatives, recall 0.90 requires TP=90. If precision were 0.05, then FP would need to be $90\times(1/0.05-1)=1{,}710$, more than the 1,000 negatives available. That recall-precision pair cannot occur on this dataset; feasibility depends on the fixed class totals.

Slide 63: - Two models

Slide 63 presents a comparison between two models: M1 with precision and recall both at 0.80, and M2 with precision at 0.60 and recall at 0.90. The slide highlights that determining which model is superior depends on the specific situation, as there is no single general answer when metrics trade off against each other. This scenario illustrates Problem 4, emphasizing the complexity of comparing models using two distinct evaluation metrics.

Slide 64: - Two models

The two models trade precision against recall: M1 has 0.80 for each, while M2 has precision 0.60 and recall 0.90. Their F1 scores are 0.80 and 0.72, respectively, using the harmonic mean. F1 favors M1 here, but the application may prefer M2 if finding more positives matters more than the extra false alarms. The slide does not give their accuracies.

Slide 65: - All metrics are based on a finite set

Metrics estimated from a finite test set have sampling uncertainty. The slide gives approximate confidence intervals of estimate ± z×standard error, with different denominators for accuracy, precision, and recall because each is based on a different number of relevant examples. A wider interval means less precision in the estimate; the z value sets the confidence level.

Slide 66: - All metrics are based on a finite set

For an approximately normal estimate, a 95% interval uses z≈1.96 standard errors on either side of the measured score. This is an interval procedure with repeated-sampling coverage, not a statement that a particular fixed parameter has a 95% probability of moving inside the observed interval. Small counts or boundary scores can make the normal approximation unreliable.

Slide 67: - Practice: Accurate probabilities are calibrated probabilities

Accurate probabilities are defined as calibrated probabilities, where the predicted likelihood of an event matches its actual observed frequency. This concept is visualized on a graph plotting predicted probability against actual probability, where perfect calibration occurs when all predictions fall along the diagonal line from (0.0, 0.0) to (1.0, 1.0).

Slide 68: - Practice: Accurate probabilities are calibrated probabilities

A calibration plot compares predicted probability (horizontal axis) with observed event frequency (vertical axis). Perfect calibration lies on y=x. A curve below that diagonal means outcomes occur less often than predicted: the model is overconfident. Calibration concerns the meaning of probability scores, not just whether the class ranking is good.

Slide 69: - Practice: Accurate probabilities are calibrated probabilities

The second build adds underconfidence. A curve above y=x means outcomes occur more often than predicted, so the model understates risk and is underconfident. The slide’s dashed curves show illustrative deviations; actual calibration is estimated by grouping predictions and comparing each group’s mean prediction with its observed frequency.

Slide 70: - Many classifiers are NOT well calibrated

Isotonic regression and beta calibration can post-process classifier scores. The slide’s Brier score is the mean squared difference between the true class indicator(s) and predicted probabilities across examples/classes; lower is better. It is a proper overall probability score, not a pure calibration-only measure, so interpret it alongside calibration plots and the task’s ranking needs.

Slide 71: - Root mean squared error: □□□□=

For real-valued outcomes, RMSE takes the square root of mean squared error and penalizes large errors more heavily; MAE averages absolute error and is less sensitive to outliers. Relative error divides squared prediction error by the squared error from always predicting the training-set mean: 0 is perfect and 1 matches that mean baseline.

Slide 72: Evaluation methodology

Section divider for evaluation methodology. The next section asks whether a score estimates performance on new examples, then develops train/test splits, cross-validation, representativeness, feature handling, and hyperparameter selection.

Slide 73: - Predict the price of stocks

Slide 73 illustrates diverse applications for deploying predictive models, including forecasting stock prices, assessing cancer risk, classifying emails as spam, and determining if a song fits a specific playlist. These examples demonstrate the broad utility of machine learning across financial, medical, communication, and entertainment domains. The slide concludes by framing these scenarios within the context of preparing to deploy a model.

Slide 74: - Predict the price of stocks

The question on this slide is what makes a model usable across the examples shown. The key property is generalization: performance should hold on new data drawn from the deployment setting, not just the examples used to fit or select the model. A metric only has practical meaning when its test data and error priorities match that setting.

Slide 75: Methodology: Generalization

Section divider: generalization. A model generalizes when patterns learned from training examples support accurate predictions on genuinely unseen examples from the target population.

Slide 76: - Idea 1: Evaluate the performance on the data used to learn the model (training data)

The plotted example has training F1=0.818 and error rate 0.154, but on independent test data F1=0.714 and error rate 0.286. Training scores are optimistic because the model was fitted on those examples; the gap does not prove the model merely memorized, but it shows why evaluation data must play no part in fitting or selection.

Slide 77: - Dilemma: need both a large training set and a large test set

There is a data-allocation trade-off: more training examples can improve the learned classifier, while more independent test examples make the generalization estimate more stable. A single split can be costly when labeled data is scarce, motivating cross-validation.

Slide 78: - Dilemma: need both a large training set and a large test set

The dilemma is practical as well as statistical: collecting labels can be expensive, and industry data may not create immediate value. Reusing different folds for training and testing lets each example contribute to an estimate, though it does not create new independent information or replace a final untouched test set when one is needed.

Slide 79: - Dilemma: need both a large training set and a large test set

Benchmark datasets let researchers compare methods on shared data, but the slide notes that repeated methodological development can still be costly and that industry labels are often a limited resource. Cross-validation reuses a small dataset for model fitting and error estimation by rotating held-out folds; it estimates the learning procedure, not a magical larger dataset.

Slide 80: - Partition examples into k disjoint sets

In cross-validation, the dataset is partitioned into k disjoint sets, or folds. This process generates k distinct training sets, where each set comprises the union of all folds except one. Consequently, every training set contains exactly $\frac{k-1}{k}$ of the original data, ensuring comprehensive model evaluation while systematically excluding a portion for testing.

Slide 81: - Five-fold cross validation with 250 examples

Five-fold cross validation partitions a dataset of 250 examples into five folds, with each fold containing 50 examples used once as the testing set while the remaining four serve for training. The process iterates through all five combinations, calculating individual accuracies that range from 0.60 to 0.80 based on correct and incorrect predictions in each iteration. The final overall accuracy is determined by aggregating the total correct predictions across all folds (175) divided by the total number of examples (250), resulting in a score of 0.70.

Slide 82: - Smaller data → More folds

More folds let each model train on a larger fraction of a small dataset; leave-one-out cross-validation trains n times on n−1 examples when n is small. Five- or ten-fold CV is a common cost/variance compromise. For very large datasets, the deck suggests a single holdout may be adequate, provided the test sample still represents deployment and remains independent.

Slide 83: - Representativeness problem:

The representativeness problem arises when all examples from an entire infrequent class are absent from the training data. In such cases, the resulting error estimate becomes meaningless because the model has never encountered that specific category during learning. This omission highlights a critical limitation where the training dataset fails to adequately reflect the true distribution of the target population.

Slide 84: - Representativeness problem:

Stratification preserves each class’s approximate proportion in every train/test fold. This reduces the chance that a rare class disappears from a split and makes fold comparisons more meaningful. It cannot fix a class absent from the whole dataset or other distribution shifts.

Slide 85: - Possible that held aside test data is not representative of new data gathered

A random holdout can still fail to represent deployment if positives and negatives were collected differently or their features were acquired under different conditions. Examples on the slide include cancer samples processed by different people or on different days, and songs whose playlist membership changes by day. The split should mimic how future data arrive and keep dependent observations together.

Slide 86: - Data is stratified before the cross validation is performed

In k-fold cross-validation, stratify before assigning folds, use each fold once as test data, then combine the held-out predictions or average fold errors for an overall estimate. This evaluates the learning method across several training sets rather than one final fitted model. If you repeatedly tweak choices in response to these results, reserve data or use nested evaluation to avoid optimistic selection bias.

Slide 87: Methodology: Manipulating the feature space

Section divider for feature-space manipulation. The following slides explain how to encode or group feature values, bin continuous measurements, and remove unhelpful features before learning.

Slide 88: - This can be for a variety of reasons:

Feature transformation changes how information is represented so it fits the learner and the task. Examples include converting categories to numeric or indicator features, grouping high-cardinality values, and discretizing continuous measures. Any transformation learned from data must be fit within training folds to avoid leakage.

Slide 89: - Convert ordinal to numeric which enables using > or <

Ordinal categories have a meaningful order, so mapping them to ordered numbers can preserve that structure. Nominal categories have no order; one-hot encoding gives each category its own indicator. Assigning arbitrary integers to nominal labels would create a false order and distances for many algorithms.

Slide 90: - Group values into categories by hand

For high-cardinality categories, group values using domain knowledge or place infrequent values in “other.” Target encoding replaces a category with a target-derived statistic: in the example, skipped/play counts give skip rates of 0.20 for ABBA, 0.40 for Green Day, 0.33 for Eminem, and 0.81 for Skrillex. Fit these statistics on training data only (and within each fold) to prevent leakage.

Slide 91: - Group values into categories by hand

Target encoding uses the target to calculate a numeric value for each category, such as a mean for a real-valued target or a conditional probability for a class. Because the label contributes to its encoded feature, computing encodings before splitting can leak held-out labels. Recompute them from each training fold and apply them to that fold’s validation/test examples.

Slide 92: - BPM values: 50 70 80 100 110 120 130 140 160 180 190 240

Binning converts continuous BPM values into categories. Equal-width bins span the same numeric range, such as [50,100), [100,150), [150,200), [200,250). Equal-height (quantile) bins aim to contain similar numbers of observations, so their numeric widths can differ, as in [50,90), [90,130), [130,170), [170,250). The choice trades interval interpretability against balanced bin counts.

Slide 93: - Feature selection: Remove features prior to learning

Feature selection involves removing or choosing specific attributes prior to model training to address scenarios where feature counts exceed data points. This process facilitates easier learning by reducing parameters, enhances model interpretability for domain insights, and improves computational efficiency through lower memory usage and faster runtimes. Additionally, eliminating irrelevant attributes prevents performance degradation, typically implemented via filter-based or wrapper-based approaches.

Slide 94: - Idea: Measure each feature’s usefulness in isolation (i.e., independent of other

Filter-based feature selection evaluates each feature's usefulness in isolation by assigning a score and selecting the top k features. This approach scales efficiently to large datasets because it assesses features independently without considering their combined effects. However, it risks missing critical feature interactions, such as height and weight jointly predicting volume, and may select redundant features that provide overlapping information.

Slide 95: - Metric that computes a “correlation”

Filter-based feature selection assigns scores to features to select the top performers using two primary approaches: metrics computing correlation or single-variable classifiers. Correlation metrics include Information Gain from information theory and the Pearson correlation coefficient between features and targets. Alternatively, a single-variable classifier trains on one variable at a time-using a decision stump for discrete data or thresholding for continuous data-and selects features based on their individual predictive performance.

Slide 96: - Feature selection = search

Wrapper feature selection searches over feature subsets: a state is a selected set, an operator adds or removes a feature, and the scoring function measures a model trained with that set (usually on validation data). Forward search starts empty and adds the best next feature; backward search starts with all features and removes the best next one. Greedy steps can stop at a local choice and do not guarantee a globally optimal subset.

Slide 97: Forward wrapper-based feature selection

The forward-search diagram starts with no features and a 50% baseline validation score. At step 1, adding Energy scores 72%, better than adding BPM (62%), Speechiness (58%), or Instrumental (64%). From Energy, adding Instrumental reaches 84%, beating the other displayed two-feature candidates; the illustrated selected subset is Energy + Instrumental. These numbers are examples of candidate validation scores, not universal results.

Slide 98: Backward wrapper-based feature selection

The backward-search diagram begins with all four features and a 78% validation score. Removing BPM gives the best first score (82%); from that subset, removing Speechiness reaches 84%, leaving Energy + Instrumental. Each branch compares one-feature removals at that step; this is greedy backward elimination, not an exhaustive search of all subsets.

Slide 99: - Forward

Forward selection is cheap initially and can build a compact model, but it may miss a useful combination if neither feature helps alone. Backward elimination can consider interactions while all features are present, but it is expensive with many features and is still a greedy search. Neither strategy guarantees the globally best subset.

Slide 100: Methodology: configuring the pipeline

Section divider for configuring the learning pipeline. This includes choosing feature transformations, a model and its hyperparameters, and an evaluation procedure; the next slides focus on tuning without using the final test set for selection.

Slide 101: - How should I set the value of k in kNN?

The value of k in k-NN is a hyperparameter: it is chosen by comparing configurations, not learned as an ordinary model weight. Fit each candidate using training data and compare it on validation data. Keep a separate test set untouched until the configuration is fixed, or use cross-validation inside the training portion when one validation split is too noisy.

Slide 102: - Add an extra split: Training - Validation - Test

Use three distinct roles: training data fits the candidate model, validation data selects among hyperparameters or pipelines, and the test data estimates final performance after selection. Repeatedly choosing the best validation result can overfit that validation set, which is why it must not be reported as an unbiased final score.

Slide 103: - Single validation set can be unreliable

A single validation split can give a noisy ranking of configurations. Cross-validation rotates validation folds within the training portion; choose the configuration with the best average, then refit it on all training data. The independent test set remains separate for final evaluation.

Slide 104: - Single validation set can be unreliable

This diagram shows internal cross-validation: the outer test set stays fixed, while folds within the training portion take turns as validation data for model/configuration selection. After choosing the configuration, refit on the full training portion and evaluate once on the held-out test set. This uses one external test split.

Slide 105: - Single validation set can be unreliable

Nested cross-validation adds an outer loop. In each outer fold, the held-out outer fold estimates performance; inside the remaining data, inner cross-validation selects hyperparameters. Repeat across outer folds to estimate the whole selection procedure. This costs more computation but avoids using the same folds both to tune and to claim an unbiased evaluation.

Slide 106: - Problem: Often have to tune multiple parameters:

Tuning multiple hyperparameters, such as the number, depth, and width of trees in random forests, presents a significant challenge that is best addressed by framing it as a search problem. Two primary approaches are grid search, which exhaustively tries all possible value combinations, and random search, which samples values from the space and is often surprisingly effective.

Slide 107: - Problem: Often have to tune multiple parameters:

Tuning multiple hyperparameters, such as tree depth or ensemble size in random forests, is treated as a search problem where grid search exhaustively tests all combinations within defined ranges. Random search often proves more effective by sampling parameter sets, as it explores more distinct values for the few critical hyperparameters that strongly influence performance within the same computational budget. While this approach may not guarantee the absolute optimal configuration, it efficiently identifies high-performing parameters and can be automated through optimization learning techniques.

Slide 108: Automated Hyperparameter Tuning: -AutoWeka or

Auto-Weka and auto-sklearn automate search over specified algorithms, preprocessing choices, and hyperparameter values. The diagram treats each setting θ as a black-box candidate, evaluates it, and feeds the validation score to an optimizer (here, Bayesian optimization) that selects the next setting to try. Automation still needs a defined search space and an evaluation design that protects held-out data.

Slide 109: - Decision trees

This slide outlines the critical hyperparameters that require tuning across various machine learning models. For decision trees and ensembles, key settings include leaf minimums, tree depth, ensemble size, and random forest specifics like `max_features`. The list extends to kNN's `k` value, linear and logistic regression regularization types (L1, L2) with parameter λ, and the choice of feature selection.

Slide 110: - Need a reasonable spread of values:

Choose a spread of plausible values rather than many nearly identical ones. For parameters whose effects are multiplicative or span orders of magnitude, log-spaced candidates cover the range efficiently; the slide's 1, 5, 25 example spans a much wider range than 1, 2, 3. The search range still needs sensible lower and upper bounds.

Slide 111: - Information from outside the training set leaks into the training data (i.e., test

Data leakage occurs when information that should be unavailable at prediction time affects fitting or evaluation. The slide lists global feature selection/target encoding before cross-validation, using future data to predict the past, incorrect timestamps, and splitting examples from the same entity across folds. Fit preprocessing inside each training fold, respect time order, and split at subject/entity level when observations are dependent.

Slide 112: Questions?

The lecture closes with questions. The key review sequence is: define the task and error costs; inspect class balance and choose a suitable metric; estimate generalization with independent or properly nested data; and keep every data-dependent transformation inside the training/evaluation procedure.

Key points to remember

- Accuracy is sensitive to class proportions; confusion matrices expose the error types that one aggregate score hides.
- Choose thresholds and metrics with class prevalence and false-positive/false-negative costs in mind.
- ROC and PR curves summarize threshold trade-offs; PR precision changes with prevalence, and feasible PR points are constrained by the fixed dataset.
- Calibration measures whether predicted probabilities match observed frequencies; ranking quality and probability quality are different properties.
- Use independent data or cross-validation to estimate generalization. Keep preprocessing, feature selection, and target encoding inside each training fold.
- Use validation or inner cross-validation to select features and hyperparameters; reserve outer/test data for evaluation.